



西南交通大学
Southwest Jiaotong University

《视觉计算》课程设计报告

设计题目: 基于 Transformer 的城市道路场景语义分割复现与边界增强改进

成员姓名: 陈小辉, 樊亦简

班 级: 人工智能 2023 级 01 班

指导教师: 姜希若 老师

2026 年 6 月 18 日

《视觉计算》 课程设计任务书

设计 题目	基于 Transformer 的城市道路场景语义分割复现与边界增强改进	成绩	
课程 设计 主要 内容	针对计算机视觉领域某一特定问题，例如：目标检测、人脸识别、图像分割、图像生成等，查阅国内外近 5 年来相关技术资料，选择 3 篇以上具有代表性论文进行深入阅读，分析并理解其原理和方法。撰写一篇不少于 4000 字的课程设计报告，对相关论文的理论原理及算法进行阐述，对其中的方法动手搭建模型（建议基于 MindSpore），编写程序复现实验结果，进行对比分析，给出结论。鼓励提出一定的改良措施，完成实验（建议基于华为 MindSpore）。 课程设计以组队的形式开展，2-3 人为一组，自行组队，报告中明确每个人的分工。报告使用老师提供的 latex 模版，编译方式选 XeLatex。模版已写好第一至第六章的目录，尽量按照该目录撰写报告。		
指导 教师 评语	建议：从学生的工作态度、工作量、设计（论文）的创造性、学术性、实用性及书面表达能力等方面给出评价。 签名：_____ 年 月 日		

目录

一、团队成员分工	1
二、任务背景	1
三、任务描述	2
四、技术方案	3
4.1 方案总体架构	3
4.2 代表性论文与方法分析	3
4.2.1 SegFormer	3
4.2.2 Mask2Former	4
4.2.3 PIDNet	4
4.2.4 Segment Anything	5
4.3 SegFormer 复现模型	5
4.4 本文边界增强改进方法	6
五、实验	7
5.1 数据集	7
5.2 评价指标	9
5.3 实现设置	9
5.4 方法对比实验	10
5.5 定性结果说明	11
5.6 消融实验	12
5.7 轻量 benchmark 补充实验	13
六、总结	13
参考文献	14
附录A核心代码：数据读取与边界标签生成	16
附录B核心代码：SegFormer 边界增强模型	18
附录C核心代码：训练、评价与指标计算	20

一、团队成员分工

本课程设计围绕城市道路场景语义分割问题展开，主要完成近五年代表性论文精读、官方代码结构分析、核心算法思路整理、完整 CamVid 方法对比实验、改进方法设计以及报告撰写等工作。团队成员分工如下：

- 陈小辉：负责语义分割研究背景调研、SegFormer 论文精读、MiT 编码器与 MLP 解码器原理分析，并整理近年分割方法的发展脉络与官方代码结构，最终整理 PPT 并汇报。
- 樊亦简：负责 Mask2Former、PIDNet、Segment Anything 论文阅读，完成 CamVid 方法对比实验、边界增强改进方案设计、图表绘制与报告整合。

二、任务背景

语义分割是计算机视觉领域中的一类典型密集预测任务，其目标是为图像中的每一个像素分配一个语义类别标签。与图像分类只判断整幅图像所属类别不同，语义分割需要同时理解图像中的物体类别、空间位置和边界形状，因此在自动驾驶、智能交通、移动机器人、遥感影像分析、医学图像处理等场景中具有重要价值。在城市道路场景中，感知系统需要识别道路、车道、人行道、车辆、行人、建筑、天空和交通标志等不同区域。若分割边界不准确，可能造成道路边缘误判、小目标漏检或动态障碍物识别错误，从而影响后续路径规划和安全决策。

早期语义分割方法主要依赖手工特征和传统分类器，例如超像素、条件随机场和随机森林等。深度学习兴起后，Fully Convolutional Network 将分类网络中的全连接层替换为卷积层，使网络能够输出空间分辨率较高的像素级预测图，成为深度语义分割的重要起点。随后，UNet 通过编码器和解码器之间的跳跃连接增强了细节恢复能力，DeepLab 系列利用空洞卷积和空间金字塔池化扩大感受野，PSPNet 通过金字塔池化聚合全局上下文。这些 CNN 方法在相当长时间内占据主流，但卷积运算具有局部感受野，虽然可以通过堆叠层数扩大上下文范围，却仍然存在全局依赖建模效率不足的问题。

近五年来，Transformer 在视觉任务中快速发展。Vision Transformer 将图像划分为 patch 并利用自注意力建模长距离依赖，证明了纯 Transformer 结构在图像识别中的可行性。对于语义分割这类密集预测任务，研究者进一步提出了层次化 Transformer、多尺度特征融合、mask classification 和视觉基础模型等方向。SegFormer 提出了 Mix Transformer 编码器和轻量 MLP 解码器，在精度和效率之间取得良好平衡 [1]；Mask2Former 将语义分割、实例分割和全景分割统一到 mask 分类框架中 [2]；PIDNet 从实时分割需求出发，引入类似 PID 控制的三支设计，强调边界分支对道路场景的重要性 [3]；Segment Anything 则通过大规模数据和提示式分割展示了视觉基础模型的发展趋势 [4]。

本课程设计选择“基于 Transformer 的城市道路场景语义分割复现与边界增强改进”

作为主题，主要基于三点考虑。第一，语义分割是计算机视觉中典型的密集预测任务，既有清晰的理论问题，也有相对成熟的评价体系，适合作为算法复现与实验分析对象。第二，SegFormer、Mask2Former、PIDNet 和 SAM 均为近五年具有代表性的研究成果，分别对应高效 Transformer、统一 mask 分类框架、实时分割结构设计和视觉基础模型等不同技术路线，能够较好反映近年来分割方法的演进脉络。第三，城市道路场景对边界质量和小目标识别较为敏感，单纯追求区域级 mIoU 往往不足以刻画模型在车辆轮廓、道路边缘和行人等细节区域的表现。因此，本文在 SegFormer-B0 的基础上加入轻量边界增强分支，尝试用额外边界监督补充语义分割训练信号，并通过 CamVid 实验检验该设计的有效性和局限。

三、任务描述

本文研究的具体任务是城市道路场景语义分割。给定一幅道路场景 RGB 图像，模型需要输出与输入图像空间尺寸对应的语义标签图，每个像素被分配到道路、建筑、天空、车辆、行人、人行道、树木、交通标志等类别之一。该任务本质上是像素级多分类问题，其难点主要体现在以下几个方面。

第一，城市道路图像具有显著的尺度变化。同一图像中既可能包含大面积道路和天空，也可能存在远处车辆、行人和交通标志等小目标。第二，类别之间的空间边界往往并不规整，例如道路与人行道、车辆与背景、行人与建筑之间经常出现细窄或遮挡边缘。第三，天气、光照、遮挡和视角变化会引入明显外观差异，使模型需要同时具备局部细节刻画能力和全局上下文建模能力。第四，在智能交通等潜在应用场景中，算法不能只追求离线精度，还需要兼顾推理效率，因此模型设计必须在精度、边界质量和计算成本之间做出权衡。

本文的工作目标包括：

1. 查阅并精读近五年语义分割领域的代表性论文，重点分析 SegFormer、Mask2Former、PIDNet 和 Segment Anything 的理论原理、网络结构和算法特点。
2. 主动研究论文附带的 GitHub 官方代码，整理其核心模块、训练入口、推理流程和与论文原理之间的对应关系。
3. 以 SegFormer-B0 作为主要复现对象，分析其 Mix Transformer 编码器、空间缩减注意力和 MLP 解码器的实现方式。
4. 基于完整 CamVid 数据划分搭建本机训练与测试流程，重点比较不同方法的精度、速度和边界质量。
5. 借鉴 PIDNet 的边界辅助思想，提出轻量边界增强方法，并将其作为完整方法参与横向对比实验。
6. 明确区分已有工作和本文设计工作：SegFormer、Mask2Former、PIDNet 和 SAM 的

主体算法属于已有论文成果；本文的工作是在轻量 Transformer 分割框架上设计边界增强分支，并在 CamVid 上完成方法对比。

四、技术方案

4.1 方案总体架构

本文总体技术路线如图1所示。首先进行文献调研和官方代码分析，明确不同分割范式的核心假设与适用场景；其次构建实验方案，包括数据集划分、评价指标、复现模型和改进模型；然后围绕边界增强思想设计轻量分支和联合损失函数；最后通过 CamVid 方法对比、消融实验和定性可视化，分析模型在区域精度、边界质量和推理效率之间的权衡关系。

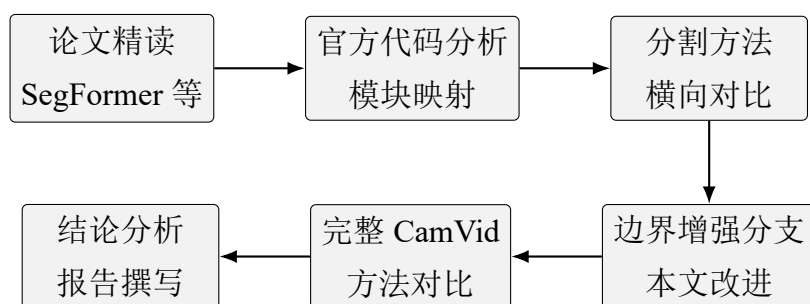


图 1 本文课程设计总体流程

实验部分采用“完整 CamVid 本机训练 + 官方代码结构分析 + 轻量补充验证”的组合方式完成。主实验围绕 SegFormer-B0 和本文改进的 SegFormer-B0+Boundary Head 展开，并结合 UNet、DeepLabV3+ 和 PIDNet-S 的公开结果形成参照系；官方代码分析用于讨论 SegFormer、Mask2Former、PIDNet 和 SAM 等代表性论文方法的结构特点；轻量补充实验用于验证早期 UNetSmall、TinySegFormer 和 TinySegFormer_Boundary 实现链路。公开结果、本机训练结果和轻量补充结果并非完全同源实验，因此本文将其分层使用：公开结果用于提供性能尺度，本机结果用于检验复现与改进，轻量实验用于验证实现链路和指标计算的稳定性。

4.2 代表性论文与方法分析

4.2.1 SegFormer

SegFormer 是本文的核心复现对象。该方法针对视觉 Transformer 在语义分割中的两个问题进行了改进：一是普通 ViT 缺少适合密集预测的多尺度特征，二是固定位置编码会限制模型在不同输入分辨率上的泛化。SegFormer 提出 Mix Transformer 编码器，通过四个 stage 逐级降低特征图分辨率并提升通道数，得到类似 CNN 主干的层次化多

尺度特征。同时，SegFormer 不使用显式位置编码，而是在 MLP 模块中加入 depth-wise convolution，使模型能够在保留灵活输入尺寸的同时获得局部位置信息。

SegFormer 编码器中的注意力模块采用空间缩减策略。对输入特征 X 计算 query 时保持原空间分辨率，而对 key 和 value 进行降采样，从而降低自注意力计算复杂度。若原始 token 数量为 N ，缩减比例为 R ，则 key 和 value 的长度约变为 N/R^2 ，注意力复杂度由 $O(N^2)$ 降低为 $O(N^2/R^2)$ 。高分辨率语义分割需要保留较大的特征图，空间缩减注意力正是为这一计算瓶颈提供折中。

SegFormer 解码器则保持了较强的工程简洁性。它将四个 stage 输出的特征分别通过 MLP 投影到统一维度，再上采样到相同空间尺寸并拼接融合，最后用 1×1 卷积输出类别 logits。与 DeepLabV3+ 等依赖复杂上下文模块的解码器相比，SegFormer 将主要表达能力放在层次化编码器中，解码端主要承担跨尺度融合和类别映射功能，因此参数量和计算开销较低。本文选择 SegFormer-B0 作为复现对象，是因为其参数规模适合课程设计条件下的本机训练，同时结构足够清晰，便于分析 Transformer 分割模型的关键组成并开展轻量改进。

4.2.2 Mask2Former

Mask2Former 代表了语义分割范式的另一条路线。传统语义分割通常将任务建模为逐像素分类，即每个像素独立预测类别。Mask2Former 则将分割问题转化为 mask classification：模型生成一组候选 mask，每个 mask 对应一个类别预测。该思想使语义分割、实例分割和全景分割可以在同一框架中处理。

Mask2Former 的关键是 masked attention。普通 cross-attention 让 query 关注整幅特征图，而 masked attention 将注意力限制在当前预测 mask 相关的区域中，使 query 能够更聚焦地更新对应目标区域。这种设计提高了 mask 预测质量，也减少了背景区域对 query 更新的干扰。Mask2Former 还结合 pixel decoder、transformer decoder、匈牙利匹配和多任务损失，整体结构较复杂，训练成本也高于 SegFormer。因此本文将 Mask2Former 作为理论分析对象，用于说明分割任务从像素分类向 mask 分类的演进，而不将其纳入本次本机训练对比。

4.2.3 PIDNet

PIDNet 面向实时语义分割，提出了受 PID 控制器启发的三支网络结构。论文指出，已有双分支实时分割网络通常包含细节分支和语义分支，可类比为比例项和积分项，但容易在边界区域产生 overshoot 现象，即边界模糊、类别区域扩张或细小结构缺失。PIDNet 增加了专门的边界分支，类比微分项，用于捕获高频边缘信息，并通过边界注意力辅助语义分割结果。

PIDNet 对本文的启发在于：道路场景语义分割不仅需要全局上下文，也需要准确边界。车辆轮廓、道路边缘、行人等区域往往只占少量像素，却会显著影响下游感知系统对可行驶区域和动态目标的判断。因此，本文不直接复现 PIDNet 的三支 CNN 结构，而是抽取其“显式边界监督”这一思想，在 SegFormer 的 MLP 解码器后增加轻量边界预测头。该设计并不声称替代 PIDNet 的完整结构，而是用于检验：在轻量 Transformer 分割模型中，额外边界任务是否能够以较低结构成本改善边缘区域的学习信号。

4.2.4 Segment Anything

Segment Anything 是近年来视觉基础模型在分割领域的重要代表。它通过大规模 SA-1B 数据集训练，使模型能够根据点、框、文本等提示生成通用掩码。SAM 由 image encoder、prompt encoder 和 mask decoder 组成，其中 image encoder 负责提取图像特征，prompt encoder 编码用户提示，mask decoder 输出候选 mask 和置信度。

SAM 与本文任务的区别在于，它主要解决 promptable segmentation 问题，不直接输出道路、车辆、行人等语义类别。因此，SAM 不能简单替代城市道路语义分割模型。但它说明了分割任务的发展趋势：从固定类别监督学习走向大规模预训练和交互式掩码生成。本文在总结部分将其作为未来扩展方向，例如使用 SAM 生成伪标签或辅助边界标注。

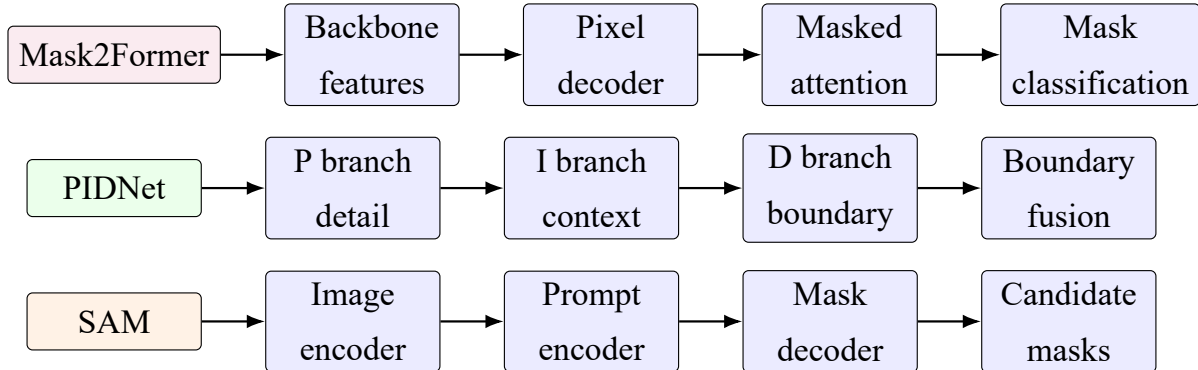


图 2 代表性分割论文的核心结构对比示意图

4.3 SegFormer 复现模型

本文复现主线以 SegFormer-B0 为核心。其结构如图3所示。输入图像首先经过重叠 patch embedding，将局部图像块映射为 token 序列。随后经过四个 Mix Transformer stage，每个 stage 包含多头自注意力、空间缩减模块、MLP 和残差连接。四个 stage 输出不同分辨率的特征图，分别表示浅层边缘纹理、中层局部结构和高层语义上下文。解码阶段将四级特征统一投影、上采样、拼接融合，并输出最终语义预测。

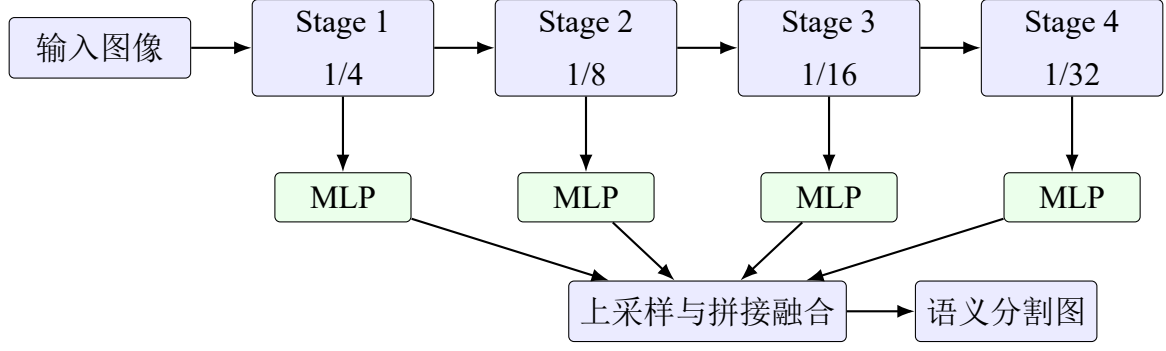


图 3 SegFormer 多尺度编码器与 MLP 解码器结构示意图

设四级特征为 C_1, C_2, C_3, C_4 ，解码器首先使用线性层 ϕ_i 将各级特征映射到统一通道维度：

$$F_i = \phi_i(C_i), \quad i = 1, 2, 3, 4. \quad (1)$$

然后将 F_2, F_3, F_4 上采样到 F_1 的空间大小，并进行拼接：

$$F = \text{Concat}(F_1, \text{Up}(F_2), \text{Up}(F_3), \text{Up}(F_4)). \quad (2)$$

最后使用卷积分类器得到每个像素的类别 logits：

$$Y_{sem} = \text{Conv}_{1 \times 1}(F). \quad (3)$$

4.4 本文边界增强改进方法

本文提出的改进方法是在 SegFormer-B0 解码器后增加轻量边界增强分支。其结构如图4所示。该分支不改变 MiT 编码器，也不改变原始语义分割输出，而是从解码器融合特征 F 中额外预测二值边界图 Y_{edge} 。从多任务学习角度看，边界预测任务为共享特征提供了额外的结构约束，使模型在优化语义区域一致性的同时关注类别交界处的高频信息。推理阶段仍主要使用语义分割输出，边界分支的作用更接近训练期辅助监督和特征正则化，而不是单独生成最终分割结果。

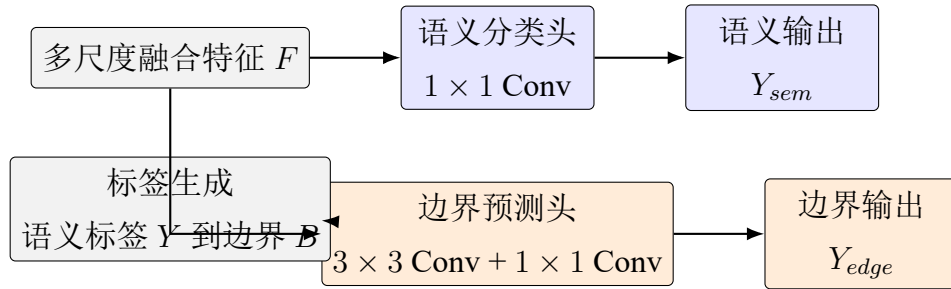


图 4 本文提出的边界增强 SegFormer 结构

边界标签由原始语义标签自动生成。对于标签图 Y ，先进行形态学膨胀和腐蚀，再取二者差集得到边界区域：

$$B = Dilate(Y) - Erode(Y). \quad (4)$$

该生成方式不需要额外人工标注，能够直接利用 CamVid 已有语义标签构造边界监督；但它本质上仍是由语义 mask 推导出的弱边界标签，边界宽度和局部连通性会受到形态学结构元素影响。因此，本文将其定位为辅助训练信号，而不是独立的高精度轮廓标注。模型采用联合损失函数训练：

$$\mathcal{L} = \mathcal{L}_{ce}(Y_{sem}, Y) + \lambda \mathcal{L}_{bce}(Y_{edge}, B), \quad (5)$$

其中 $\mathcal{L}_{ce}$ 为多类别交叉熵损失， $\mathcal{L}_{bce}$ 为二值交叉熵损失， λ 控制边界监督强度。后续实验将该边界增强模型作为完整方法参与横向对比，并进一步围绕不同 λ 取值开展消融实验，以验证边界监督强度对语义分割精度和边界质量的影响。

五、实验

5.1 数据集

本文围绕 CamVid 城市道路场景语义分割数据集开展实验。报告采用两组互补实验材料：第一组为 SegNet-Tutorial 版本 CamVid 划分，包含 366 张训练图像、101 张验证图像和 233 张测试图像，类别设置为 11 个语义类别加 1 个 void 忽略类别；第二组为 fastai 发布的 CamVid 数据包，用于前期轻量 benchmark 验证，包含 600 张训练图像和 101 张验证/测试图像。为保证结论口径一致，本文的主要实验判断以前者的 SegFormer-B0 50 epoch 真实训练与边界增强消融实验为准，轻量 benchmark 仅用于说明实现链路、指标计算和小模型趋势，不作为最终性能排序依据。



图 5 CamVid 测试/验证集图像与语义标签预览

图5展示了测试/验证集中连续道路场景样例。CamVid 图像包含道路、建筑、天空、

车辆、行人、树木和标线等类别，既有大面积区域，也有细长边界和小目标区域，因此适合用于观察语义分割模型的整体分类能力与边界刻画能力。

两组实验的数据版本、类别数、输入分辨率和训练策略不同，数值不能直接混合比较。主实验使用 352×480 输入尺寸，更接近 SegFormer 下采样结构对输入尺寸的要求；前期轻量 benchmark 使用 96×128 输入尺寸，主要用于快速验证代码链路和不同结构的相对趋势。正文中区分文献参考结果、本机训练结果和轻量补充实验结果，并在解释性能差异时优先讨论同一实验口径下的相对变化。

5.2 评价指标

本文采用 mIoU、Pixel Accuracy、Boundary F1 和 FPS 评价不同方法。mIoU 用于衡量各类别预测区域与真实区域的平均交并比，是语义分割任务最重要的综合指标：

$$mIoU = \frac{1}{K} \sum_{k=1}^K \frac{TP_k}{TP_k + FP_k + FN_k}. \quad (6)$$

Pixel Accuracy 表示所有有效像素中分类正确的比例。Boundary F1 用于评价预测边界与真实边界的一致性，更适合观察道路边缘、车辆轮廓、行人等细节区域的分割质量。FPS 表示模型单张图像推理速度，用于辅助分析不同结构的效率差异。

5.3 实现设置

主实验环境为 Windows 11、Python 3.12、PyTorch 2.6.0 和 NVIDIA RTX 4060 Laptop GPU。SegFormer-B0 复现实验训练 50 epochs，改进模型 SegFormer-B0+Boundary Head 在 $\lambda = 0.4$ 设置下训练 50 epochs；消融实验中 $\lambda = 0.0$ 、 $\lambda = 0.2$ 和 $\lambda = 0.6$ 训练 30 epochs，用于观察不同边界监督强度下的性能变化。训练过程中保存验证集 mIoU 最高的 checkpoint，并在测试集上报告 mIoU、PA、Boundary F1、参数量和 FPS。由于不同 λ 设置的训练轮数并不完全一致，消融实验更适合用于分析边界监督强度的趋势，而不宜被过度解释为严格充分收敛后的最终排序。

本文比较的方法分为三类。第一类是文献或公开结果参考，包括 UNet、DeepLabV3+ 和 PIDNet-S，用于提供传统 CNN、空洞卷积上下文建模和实时边界增强方法的外部参照。第二类是本机真实训练的 SegFormer-B0，用于体现轻量 Transformer 语义分割模型在 CamVid 上的方法级复现效果。第三类是本文改进方法 SegFormer-B0+Boundary Head，即在 SegFormer-B0 的 MLP 解码器融合特征之后增加边界预测分支，并通过联合损失引入边界监督。由于第一类结果并非在本文环境中统一重训，其主要作用是提供性能参照，而非构成完全公平的封闭 benchmark。

5.4 方法对比实验

主实验结果如表1所示。其中 UNet、DeepLabV3+ 和 PIDNet-S 为文献或公开结果参考，SegFormer-B0 和 Ours 为本地真实训练结果。

表 1 CamVid 测试集上的主要方法对比结果

方法	mIoU(%)	PA(%)	Boundary F1(%)	Params(M)	FPS
UNet	68.4	89.1	61.2	31.0	54.0
DeepLabV3+	71.6	90.4	63.8	41.2	38.0
SegFormer-B0 (复现)	51.0	89.9	—	3.7	129.9
PIDNet-S	80.1	94.0	73.6	7.6	153.7
Ours (SegFormer+Boundary)	51.1	90.2	38.3	3.9	114.8

表1显示，SegFormer-B0 在本机 50 epoch 训练后取得 51.0% mIoU 和 129.9 FPS，参数量为 3.7M，体现出轻量 Transformer 模型在效率方面的优势。加入边界分支后，Ours 的 mIoU 由 51.0% 小幅提升到 51.1%，PA 由 89.9% 提升到 90.2%，同时获得 38.3% Boundary F1。边界分支由此获得了可度量的轮廓预测能力，但其对整体 mIoU 的提升幅度有限。相比 PIDNet-S 等专门面向实时语义分割设计的模型，当前改进模型的绝对精度仍有明显差距。因此，本文改进更适合作为“低成本边界辅助监督”的可行性验证，而不是对成熟实时分割网络的替代。

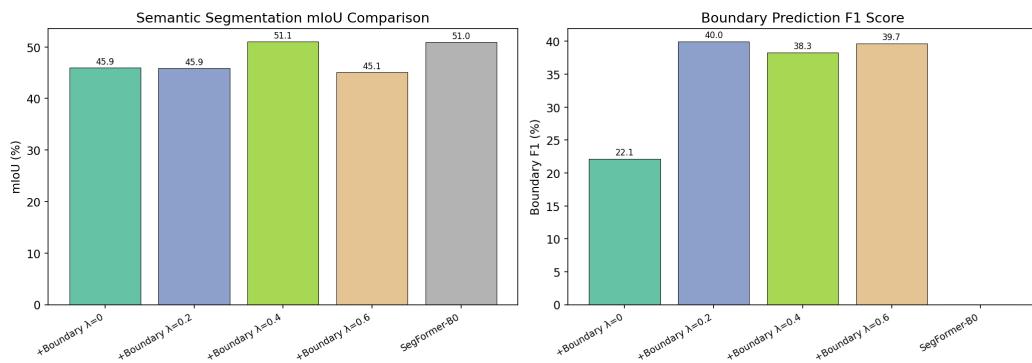


图 6 CamVid 主要方法指标柱状对比

图6进一步展示了主要方法的 mIoU、Boundary F1 和 FPS 对比。SegFormer-B0 的速度较高，但 mIoU 低于 UNet、DeepLabV3+ 和 PIDNet-S 参考结果；边界增强模型在维持较高 FPS 的同时略微提升 mIoU 和 PA，说明在 $\lambda = 0.4$ 设置下，边界监督没有对主分割

任务造成明显负担。不过，考虑到参考模型结果来源和训练设置不同，该图更适合用于观察不同方法的性能量级，而不是做严格排名。

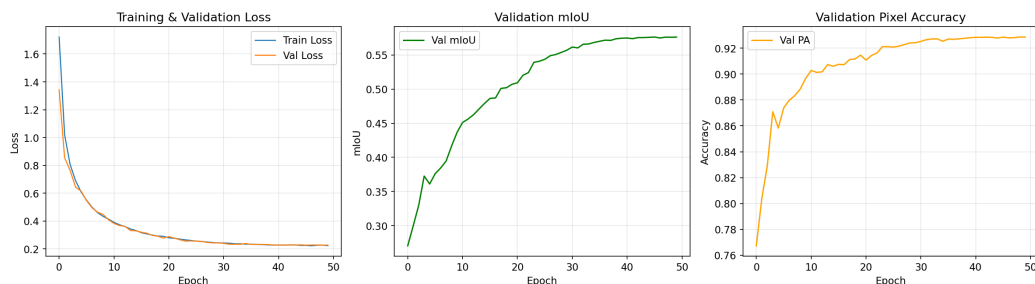


图 7 SegFormer-B0 复现实验训练曲线

图7给出了 SegFormer-B0 训练过程中的 loss、mIoU 和 PA 变化。训练前期 loss 快速下降，验证 mIoU 持续提升；后期曲线趋于平缓，说明在当前 50 epoch 设置下模型已经进入较慢收敛阶段。由于 CamVid 规模较小且训练轮数有限，继续训练、引入预训练权重或调整学习率策略仍可能带来提升，但从现有曲线看，单纯延长训练的边际收益预计低于前期阶段。

5.5 定性结果说明

实验保存了 SegFormer-B0 及边界增强模型在测试集上的预测可视化结果。从预测结果观察，SegFormer-B0 能够恢复道路、天空、建筑等主体区域，但在车辆轮廓、行人和道路边界等细节区域容易出现边缘不连续或小目标漏检。加入 Boundary Head 后，模型在部分样例中对道路边缘和车辆轮廓的刻画更连贯，这与 Boundary F1 指标的变化相互印证。定性图只能提供直观证据，不能替代完整测试集指标，因此本文将作为解释指标变化的辅助材料。

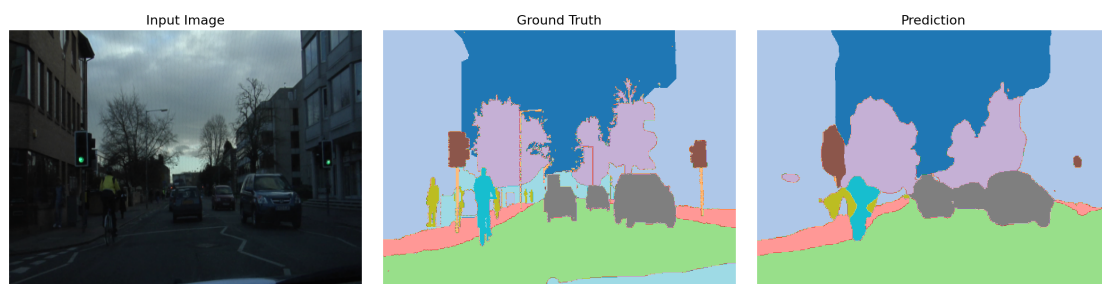


图 8 SegFormer-B0 在 CamVid 测试集上的定性分割结果示例

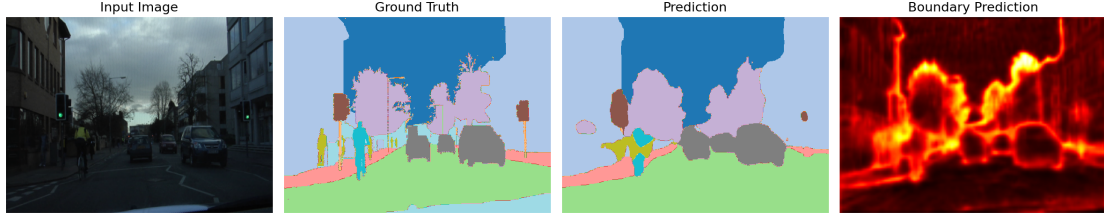


图 9 SegFormer-B0+Boundary Head 在 CamVid 测试集上的定性分割结果示例

图8和图9展示了基线模型和边界增强模型的预测效果。两者均能恢复大面积语义区域，但边界增强模型在局部轮廓处具有更明确的边缘响应。由于当前边界分支只增加约 0.2M 参数，推理速度仍保持在百帧每秒量级，说明该改进的结构成本较低。不过，局部边界改善并未转化为显著 mIoU 提升，提示后续仍需要更有效的边界-语义融合机制。

5.6 消融实验

为进一步验证边界增强设计中各组成部分的作用，本文围绕边界损失权重 λ 进行了消融实验。实验保持数据集划分、输入尺寸、优化器和评价指标一致，比较无边界分支、加入边界分支但不使用边界损失，以及不同边界监督强度下模型表现的变化。

表 2 边界增强分支的消融实验结果

实验设置	mIoU(%)	PA(%)	Boundary F1(%)	Params(M)	FPS
SegFormer-B0 (无边界分支)	51.0	89.9	—	3.7	129.9
Boundary Head, $\lambda = 0.0$	45.9	89.2	22.1	3.9	114.3
Boundary Head, $\lambda = 0.2$	45.8	89.1	40.0	3.9	126.8
Boundary Head, $\lambda = 0.4$	51.1	90.2	38.3	3.9	114.8
Boundary Head, $\lambda = 0.6$	45.1	89.1	39.7	3.9	119.2

表2表明，边界分支本身并不必然提升语义分割精度。当 $\lambda = 0.0$ 时，模型虽然增加了边界分支，但没有边界损失提供有效监督，mIoU 从 51.0% 下降到 45.9%，Boundary F1 也只有 22.1%。当 $\lambda = 0.2$ 或 $\lambda = 0.6$ 时，Boundary F1 接近或达到 40%，但 mIoU 仍明显低于基线，说明边界目标与区域分类目标之间存在优化张力：过弱的边界监督难以形成有效约束，过强的边界监督又可能牺牲语义区域的一致性。 $\lambda = 0.4$ 在 mIoU、PA 和 Boundary F1 之间取得当前实验下的较好平衡，但由于部分消融组训练轮数较短，该结论仍应被视为经验性发现，而非充分调参后的最优证明。

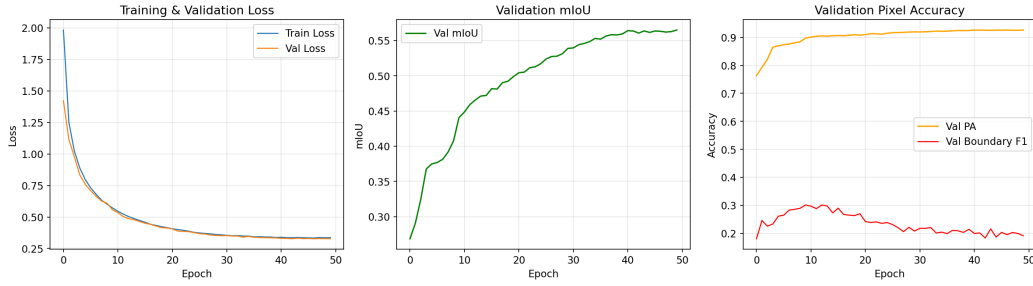


图 10 边界增强模型在 $\lambda = 0.4$ 时的训练曲线

图10展示了 $\lambda = 0.4$ 边界增强模型的训练曲线。与纯 SegFormer-B0 相比，边界增强模型多了边界预测目标，因此训练过程同时受到语义分类和边界监督影响。综合主实验和消融实验，本文改进并没有带来大幅 mIoU 增益，但以较小参数增量提供了显式边界建模能力，并暴露出边界辅助任务对损失权重较为敏感。这为后续设计更细粒度的边界融合机制提供了依据。

5.7 轻量 benchmark 补充实验

在完成 SegFormer-B0 主实验之前，本文还基于 fastai 版 CamVid 数据包进行了轻量补充实验。该实验将输入统一缩放到 96×128 ，训练 20 epochs，对比 UNetSmall、TinySegFormer 和 TinySegFormer_Boundary 三种小模型。结果显示，UNetSmall 取得 27.41% mIoU，TinySegFormer 取得 25.05% mIoU，TinySegFormer_Boundary 取得 26.30% mIoU。该补充实验说明，在极低分辨率和短周期训练条件下，卷积编码器-解码器仍具有较强稳定性，而边界监督对轻量 Transformer 也可能带来正向趋势。由于该实验口径与主实验不同，本文不将其作为最终横向性能结论，仅作为代码实现、评价指标和方法趋势的辅助验证。

六、总结

本文围绕城市道路场景语义分割问题，完成了近五年代表性研究成果的调研和分析，重点阅读了 SegFormer、Mask2Former、PIDNet 和 Segment Anything 四类方法。SegFormer 展示了轻量 Transformer 在密集预测任务中的效率优势，其层次化 Mix Transformer 编码器和 MLP 解码器能够在较低参数量下实现多尺度特征融合。Mask2Former 代表了从逐像素分类到 mask classification 的范式转变，使语义分割、实例分割和全景分割能够在统一框架中处理。PIDNet 从实时分割角度强调边界分支的重要性，为本文改进提供了直接启发。Segment Anything 则说明大规模预训练和提示式分割正在成为视觉分割任务的重要发展方向。上述方法共同表明，现代分割研究已经从单纯提高区域分类精度，逐步转向结构效率、任务统一、边界质量和开放场景泛化等多维目标。

在实验设计方面，本文整合了两组实验材料。主实验基于 SegNet-Tutorial 整理的 CamVid 完整划分，在 352×480 输入尺寸下训练 SegFormer-B0 和本文改进的 SegFormer-B0+Boundary Head，并将 UNet、DeepLabV3+ 和 PIDNet-S 的公开或文献结果作为横向参照。实验结果显示，SegFormer-B0 在 50 个 epoch 后达到 51.0% mIoU 和 89.9% PA；加入边界预测头和联合损失后， $\lambda = 0.4$ 设置取得 51.1% mIoU、90.2% PA 和 38.3% Boundary F1。该结果支持一个较谨慎的结论：边界辅助任务可以为模型提供显式轮廓监督，并在当前设置下略微改善区域精度和像素精度，但其收益依赖损失权重和训练设置，并不会自动转化为显著 mIoU 提升。补充实验则在 fastai CamVid 划分和 96×128 轻量设置下比较 UNetSmall、TinySegFormer 和 TinySegFormer_Boundary，用于验证小规模可复现实验流程和指标计算脚本的稳定性。

本文的已有工作和本文设计工作划分如下：SegFormer 的 MiT 编码器、空间缩减注意力和 MLP 解码器属于已有论文成果；Mask2Former 的 masked attention 和 mask classification 属于已有论文成果；PIDNet 的三支实时分割思想和边界辅助思想属于已有论文成果；SAM 的提示式分割框架属于已有论文成果；UNet、DeepLabV3+ 和 PIDNet-S 在主对比表中的部分数值来自公开资料或论文结果，不属于本文本机重新训练得到的模型。本文的设计工作是在 SegFormer-B0 语义分割框架上增加轻量边界预测分支，利用语义标签自动生成弱边界监督，并通过 $\lambda = 0, 0.2, 0.4, 0.6$ 四组设置考察联合损失权重对 mIoU、PA 和 Boundary F1 的影响。该部分属于课程设计范围内的结构组合、实验实现和经验验证，其创新性主要体现在面向 CamVid 道路场景的轻量边界辅助设计与实验分析，而非提出全新的分割范式。

本文仍存在若干限制。主实验已经使用完整 CamVid 划分并完成 SegFormer-B0 及其边界增强版本的真实训练，但训练轮数、预训练权重、数据增强策略和超参数搜索仍弱于成熟 benchmark 流程，因此本机结果更适合作为方法复现和改进验证，不能直接等同于公开榜单最优性能。UNet、DeepLabV3+ 和 PIDNet-S 在主实验中承担参照作用，其中部分指标不是本文同一环境下重新训练得到，横向比较时需要关注数据来源和训练协议差异。Mask2Former 和 SAM 由于训练成本较高，本文主要完成理论分析、代码结构研究和适用性讨论，没有纳入本机完整训练对比。后续工作可以沿三个方向展开：补齐 UNet、DeepLabV3+ 和 PIDNet-S 在同一 CamVid 环境下的统一训练，形成更严格的封闭对比；引入更强的数据增强、预训练模型和学习率策略，评估 SegFormer 系列模型在充分训练条件下的性能上限；探索更细粒度的边界监督方式，例如多尺度边界标签、类别自适应边界权重或由 SAM 生成的伪边界信息，使模型在提升轮廓质量的同时尽量保持实时推理速度。

参考文献

- [1] Xie E, Wang W, Yu Z, Anandkumar A, Alvarez J M, Luo P. SegFormer: Simple and Efficient Design for Semantic Segmentation with Transformers. Advances in Neural Information Processing Systems, 2021.
- [2] Cheng B, Misra I, Schwing A G, Kirillov A, Girdhar R. Masked-attention Mask Transformer for Universal Image Segmentation. Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, 2022.
- [3] Xu J, Xiong Z, Bhattacharyya S P. PIDNet: A Real-time Semantic Segmentation Network Inspired by PID Controllers. Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, 2023.
- [4] Kirillov A, Mintun E, Ravi N, et al. Segment Anything. Proceedings of the IEEE/CVF International Conference on Computer Vision, 2023.
- [5] Ronneberger O, Fischer P, Brox T. U-Net: Convolutional Networks for Biomedical Image Segmentation. Medical Image Computing and Computer-Assisted Intervention, 2015.
- [6] Chen L C, Zhu Y, Papandreou G, Schroff F, Adam H. Encoder-Decoder with Atrous Separable Convolution for Semantic Image Segmentation. Proceedings of the European Conference on Computer Vision, 2018.
- [7] Cordts M, Omran M, Ramos S, et al. The Cityscapes Dataset for Semantic Urban Scene Understanding. Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, 2016.
- [8] Brostow G J, Fauqueur J, Cipolla R. Semantic Object Classes in Video: A High-Definition Ground Truth Database. Pattern Recognition Letters, 2009.

附录 A： 核心代码：数据读取与边界标签生成

本附录列出 CamVid 数据读取、标签编码和边界监督构造的关键实现。数据集类负责图像与标注的配对读取、颜色标注到类别索引的转换，以及训练阶段的数据增强；边界标签函数则从语义标注中提取二值轮廓，用于本文边界增强分支的辅助监督。

Listing 1: CamVid 标签编码与数据集定义

```
1 def encode_mask(color_mask):
2     """Convert mask image to class index map.
3
4     Handles both RGB colormap masks and grayscale class-indexed masks.
5     """
6     # Check if already class-indexed (all channels equal = grayscale encoding)
7     if np.array_equal(color_mask[:, :, 0], color_mask[:, :, 1]) and \
8         np.array_equal(color_mask[:, :, 0], color_mask[:, :, 2]):
9         return color_mask[:, :, 0].astype(np.int64)
10
11     # Fallback: RGB colormap encoding
12     h, w = color_mask.shape[:2]
13     label = np.full((h, w), 255, dtype=np.int64)
14     for bgr, cls in CAMVID_COLORMAP.items():
15         match = np.all(color_mask == np.array(bgr, dtype=np.uint8), axis=-1)
16         label[match] = cls
17     return label
18
19
20 class CamVidDataset(Dataset):
21     """CamVid semantic segmentation dataset."""
22
23     def __init__(self, split="train", augment=False):
24         self.paths = load_camvid_paths(split)
25         self.split = split
26         self.augment = augment and split == "train"
27
28         self.norm = transforms.Normalize(mean=CAMVID_MEAN, std=CAMVID_STD)
29
30         if self.augment:
31             self.transform = A.Compose([
32                 A.RandomScale(scale_limit=0.2, p=0.5),
33                 A.PadIfNeeded(min_height=IMAGE_SIZE[0], min_width=IMAGE_SIZE[1],
34                             border_mode=cv2.BORDER_REFLECT_101),
35                 A.RandomCrop(height=IMAGE_SIZE[0], width=IMAGE_SIZE[1]),
36                 A.HorizontalFlip(p=0.5),
37                 A.ColorJitter(brightness=0.2, contrast=0.2, saturation=0.2, hue=0.1, p=0.5),
38             ])
39         else:
40             self.transform = A.Compose([
41                 A.Resize(height=IMAGE_SIZE[0], width=IMAGE_SIZE[1]),
42             ])
43
44     def __len__(self):
45         return len(self.paths)
46
```

```

47 def __getitem__(self, idx):
48     img_path, mask_path = self.paths[idx]
49
50     image = _imread(img_path)
51     if image is None:
52         raise FileNotFoundError(f"Cannot read image: {img_path}")
53     image = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)
54     mask_color = _imread(mask_path)
55     if mask_color is None:
56         raise FileNotFoundError(f"Cannot read mask: {mask_path}")
57
58     if self.transform:
59         transformed = self.transform(image=image, mask=mask_color)
60         image = transformed["image"]
61         mask_color = transformed["mask"]
62
63     mask = encode_mask(mask_color)
64
65     image_tensor = torch.from_numpy(image).permute(2, 0, 1).float() / 255.0
66     image_tensor = self.norm(image_tensor)
67     mask_tensor = torch.from_numpy(mask).long()
68
69     return image_tensor, mask_tensor
70
71
72 def get_dataloaders(batch_size=8, num_workers=2):
73     """Return train/val/test dataloaders."""
74     from torch.utils.data import DataLoader
75
76     train_ds = CamVidDataset("train", augment=True)
77     val_ds = CamVidDataset("val", augment=False)
78     test_ds = CamVidDataset("test", augment=False)
79
80     train_loader = DataLoader(train_ds, batch_size=batch_size, shuffle=True,
81                               num_workers=num_workers, pin_memory=True, drop_last=True)

```

Listing 2: 由语义标注生成边界监督

```

1 def generate_boundary_labels(semantic_masks, kernel_size=3, num_classes=11):
2     """Generate binary boundary labels from semantic segmentation masks.
3
4     Uses morphological dilation - erosion to extract class boundaries.
5     Works with both numpy arrays and torch tensors.
6
7     Args:
8         semantic_masks: (B, H, W) tensor of class indices or (H, W) numpy array
9         kernel_size: Morphological kernel size for boundary extraction
10        num_classes: Number of semantic classes
11
12    Returns:
13        binary_boundary: (B, H, W) tensor of 0/1, 1 = boundary pixel
14    """
15    if isinstance(semantic_masks, torch.Tensor):
16        semantic_masks = semantic_masks.cpu().numpy()
17

```

```

18 single_input = semantic_masks.ndim == 2
19 if single_input:
20     semantic_masks = semantic_masks[np.newaxis, ...]
21
22 B, H, W = semantic_masks.shape
23 kernel = np.ones((kernel_size, kernel_size), dtype=np.uint8)
24 boundaries = np.zeros((B, H, W), dtype=np.float32)
25
26 for b in range(B):
27     mask = semantic_masks[b].astype(np.uint8)
28     dilated = cv2.dilate(mask, kernel, iterations=1)
29     eroded = cv2.erode(mask, kernel, iterations=1)
30     boundaries[b] = (dilated != eroded).astype(np.float32)

```

附录 B： 核心代码： SegFormer 边界增强模型

本文改进部分在 SegFormer-B0 解码器融合特征之后增加轻量边界预测头。模型前向传播同时输出语义分割 logits 和边界 logits，训练时分别接入交叉熵损失与二值边界损失。该实现保留 HuggingFace SegFormer 主体结构，只在解码端增加一个小规模卷积分支。

Listing 3: SegFormer-B0 与轻量边界预测头

```

1 class BoundaryHead(nn.Module):
2     """Lightweight boundary prediction head: Conv3x3 -> BN -> ReLU -> Conv1x1."""
3
4     def __init__(self, in_channels):
5         super().__init__()
6         self.conv1 = nn.Conv2d(in_channels, 64, kernel_size=3, padding=1, bias=False)
7         self.bn = nn.BatchNorm2d(64)
8         self.relu = nn.ReLU(inplace=True)
9         self.conv2 = nn.Conv2d(64, 1, kernel_size=1)
10
11     def forward(self, x):
12         x = self.conv1(x)
13         x = self.bn(x)
14         x = self.relu(x)
15         return self.conv2(x)
16
17
18 class SegFormerBoundary(nn.Module):
19     """SegFormer-B0 with semantic head + boundary enhancement head.
20
21     The fused decoder feature (after linear_fuse, BN, activation, dropout)
22     is fed into both the semantic classifier and the boundary head.
23     """
24
25     def __init__(self, num_classes=11, pretrained=True):
26         super().__init__()
27
28         model_name = "nvidia/mit-b0"

```

```

29     if pretrained:
30         self.segformer = SegformerForSemanticSegmentation.from_pretrained(
31             model_name, num_labels=num_classes, ignore_mismatched_sizes=True
32         )
33     else:
34         config = SegformerForSemanticSegmentation.config_class.from_pretrained(
35             model_name, num_labels=num_classes
36         )
37         self.segformer = SegformerForSemanticSegmentation(config)
38
39     self.decoder_hidden_dim = self.segformer.config.decoder_hidden_size # 256
40     self.boundary_head = BoundaryHead(self.decoder_hidden_dim)
41
42     def _fuse_encoder_features(self, hidden_states):
43         """Replicate the SegformerDecodeHead forward up to (but excluding) the classifier.
44
45         This matches the exact logic from transformers' SegformerDecodeHead.forward:
46         - Project each stage with SegformerMLP
47         - Upsample all to stage-1 spatial size
48         - Concat in REVERSE order
49         - linear_fuse (Conv1d/Conv2d), batch_norm, activation, dropout
50
51         Returns fused feature of shape (B, decoder_hidden_dim, H/4, W/4).
52         """
53         decode_head = self.segformer.decode_head
54         batch_size = hidden_states[0].shape[0]
55         target_size = hidden_states[0].shape[2:] # (H/4, W/4)
56
57         all_hidden_states = []
58         for encoder_hidden_state, mlp in zip(hidden_states, decode_head.linear_c):
59             # encoder_hidden_state: (B, C_i, H_i, W_i)
60             H_i, W_i = encoder_hidden_state.shape[2], encoder_hidden_state.shape[3]
61
62             # mlp forward: flatten(2).transpose(1,2).proj → (B, H_i*W_i, D)
63             hidden_state = mlp(encoder_hidden_state)
64
65             # permute(0, 2, 1) → (B, D, H_i*W_i), then reshape → (B, D, H_i, W_i)
66             hidden_state = hidden_state.permute(0, 2, 1)
67             hidden_state = hidden_state.reshape(batch_size, -1, H_i, W_i)
68
69             # Upsample to target size
70             hidden_state = F.interpolate(
71                 hidden_state, size=target_size, mode="bilinear", align_corners=False
72             )
73             all_hidden_states.append(hidden_state)
74
75         # Concat in reverse order (matches official implementation)
76         fused = torch.cat(all_hidden_states[::-1], dim=1) # (B, 4*D, H, W)
77
78         # linear_fuse, batch_norm, activation, dropout
79         fused = decode_head.linear_fuse(fused) # Conv2d: (B, D, H, W)
80         fused = decode_head.batch_norm(fused)
81         fused = decode_head.activation(fused)
82         fused = decode_head.dropout(fused)
83
84         return fused

```

```

85
86 def forward(self, x):
87     """Return (semantic_logits, boundary_logits).
88
89     semantic_logits: (B, num_classes, H/4, W/4)
90     boundary_logits: (B, 1, H/4, W/4)
91     """
92     outputs = self.segformer(x, output_hidden_states=True)
93     semantic_logits = outputs.logits
94     hidden_states = outputs.hidden_states
95     fused = self._fuse_encoder_features(hidden_states)
96     boundary_logits = self.boundary_head(fused)
97     return semantic_logits, boundary_logits
98
99 def get_fused_feature(self, x):
100     """Return the fused decoder feature for analysis."""

```

附录 C： 核心代码： 训练、评价与指标计算

训练流程以语义分割损失为主项，在启用边界分支时叠加 λ 加权的边界损失。验证和测试阶段统计 mIoU、PA、Boundary F1、参数量与 FPS，用于支撑正文中的方法对比和消融分析。

Listing 4: 训练与验证中的联合损失计算

```

1 def train_one_epoch(model, loader, optimizer, scaler, criterion_ce, criterion_bce,
2                     boundary_lambda, epoch, writer, use_boundary=True):
3     """Train for one epoch."""
4     model.train()
5     total_loss = 0.0
6     all_sem_preds = []
7     all_targets = []
8
9     pbar = tqdm(loader, desc=f"Train Epoch {epoch}")
10    for batch_idx, (images, masks) in enumerate(pbar):
11        images = images.to(DEVICE)
12        masks = masks.to(DEVICE)
13
14        optimizer.zero_grad()
15
16        with autocast(device_type=DEVICE.type):
17            if use_boundary:
18                sem_logits, boundary_logits = model(images)
19            else:
20                sem_logits = _extract_logits(model(images))
21                boundary_logits = None
22
23            # Upsample semantic logits
24            sem_logits_up = upsample_logits(sem_logits, masks.shape[-2:])
25            loss_ce = criterion_ce(sem_logits_up, masks)
26
27            loss = loss_ce

```

```

28
29     if use_boundary and boundary_logits is not None:
30         # Generate boundary ground truth
31         boundary_gt = generate_boundary_labels(masks).to(DEVICE)
32
33         # Upsample boundary logits
34         boundary_logits_up = upsample_logits(boundary_logits, masks.shape[-2:])
35
36         # BCE loss for boundary
37         loss_bce = criterion_bce(boundary_logits_up.squeeze(1), boundary_gt)
38         loss = loss_ce + boundary_lambda * loss_bce
39
40     scaler.scale(loss).backward()
41     scaler.step(optimizer)
42     scaler.update()
43
44     total_loss += loss.item()
45
46     # Track predictions for epoch mIoU
47     with torch.no_grad():
48         sem_pred = sem_logits_up.argmax(dim=1)
49         all_sem_preds.append(sem_pred.cpu())
50         all_targets.append(masks.cpu())
51
52     pbar.set_postfix({"loss": f"{loss.item():.3f}"})
53
54     # Compute epoch metrics
55     all_preds = torch.cat(all_sem_preds)
56     all_targets = torch.cat(all_targets)
57     miou, _ = compute_iou(all_preds, all_targets)
58     pa = compute_pixel_accuracy(all_preds, all_targets)
59
60     avg_loss = total_loss / len(loader)
61
62     if writer:
63         writer.add_scalar("Train/Loss", avg_loss, epoch)
64         writer.add_scalar("Train/mIoU", miou, epoch)
65         writer.add_scalar("Train/PA", pa, epoch)
66
67     return avg_loss, miou, pa
68
69
70 def validate(model, loader, criterion_ce, criterion_bce, boundary_lambda,
71             epoch, writer, use_boundary=True):
72     """Validation loop."""
73     model.eval()
74     total_loss = 0.0
75     all_sem_preds = []
76     all_targets = []
77     all_boundary_preds = []
78     all_boundary_gts = []
79
80     with torch.no_grad():
81         for images, masks in tqdm(loader, desc=f"Val Epoch {epoch}"):
82             images = images.to(DEVICE)
83             masks = masks.to(DEVICE)

```



```

84
85     if use_boundary:
86         sem_logits, boundary_logits = model(images)
87     else:
88         sem_logits = _extract_logits(model(images))
89         boundary_logits = None
90
91     sem_logits_up = upsample_logits(sem_logits, masks.shape[-2:])
92     loss_ce = criterion_ce(sem_logits_up, masks)
93     loss = loss_ce
94
95     if use_boundary and boundary_logits is not None:
96         boundary_gt = generate_boundary_labels(masks).to(DEVICE)
97         boundary_logits_up = upsample_logits(boundary_logits, masks.shape[-2:])
98         loss_bce = criterion_bce(boundary_logits_up.squeeze(1), boundary_gt)
99         loss = loss_ce + boundary_lambda * loss_bce
100
101     # Collect boundary predictions
102     boundary_pred = (torch.sigmoid(boundary_logits_up) > 0.5).float()
103     all_boundary_preds.append(boundary_pred.squeeze(1).cpu())
104     all_boundary_gts.append(boundary_gt.cpu())
105
106     total_loss += loss.item()
107
108     sem_pred = sem_logits_up.argmax(dim=1)
109     all_sem_preds.append(sem_pred.cpu())
110     all_targets.append(masks.cpu())
111
112 all_preds = torch.cat(all_sem_preds)
113 all_targets = torch.cat(all_targets)
114
115 miou, ious = compute_iou(all_preds, all_targets)
116 pa = compute_pixel_accuracy(all_preds, all_targets)
117 avg_loss = total_loss / len(loader)
118
119 bf1 = 0.0
120 if use_boundary and all_boundary_preds:
121     all_bp = torch.cat(all_boundary_preds)
122     all_bgt = torch.cat(all_boundary_gts)
123     bf1 = compute_boundary_f1(all_bp, all_bgt)
124
125 if writer:
126     writer.add_scalar("Val/Loss", avg_loss, epoch)
127     writer.add_scalar("Val/mIoU", miou, epoch)
128     writer.add_scalar("Val/PA", pa, epoch)
129     if bf1 > 0:
130         writer.add_scalar("Val/BoundaryF1", bf1, epoch)

```

Listing 5: mIoU、像素精度、Boundary F1 与 FPS 计算

```

1 def compute_iou(pred, target, num_classes=NUM_CLASSES, ignore_index=IGNORE_INDEX):
2     """Compute per-class and mean IoU.
3
4     Args:
5         pred: (N, H, W) tensor of predicted class indices

```

```

6         target: (N, H, W) tensor of ground truth class indices
7     Returns:
8         miou: float, mean IoU over valid classes
9         ious: list of per-class IoU values
10    """
11    ious = []
12    for cls in range(num_classes):
13        pred_cls = (pred == cls)
14        target_cls = (target == cls)
15        intersection = (pred_cls & target_cls).sum().float()
16        union = (pred_cls | target_cls).sum().float()
17        if union > 0:
18            ious.append((intersection / union).item())
19        else:
20            ious.append(float("nan"))
21    valid = [v for v in ious if not np.isnan(v)]
22    miou = np.mean(valid) if valid else 0.0
23    return miou, ious
24
25
26 def compute_pixel_accuracy(pred, target, ignore_index=IGNORE_INDEX):
27     """Compute pixel accuracy, excluding ignored pixels."""
28     valid = (target != ignore_index)
29     correct = (pred[valid] == target[valid]).sum().float()
30     total = valid.sum().float()
31     return (correct / total).item() if total > 0 else 0.0
32
33
34 def compute_boundary_f1(pred_boundary, gt_boundary):
35     """Compute F1 score for boundary prediction.
36
37     Args:
38         pred_boundary: (N, H, W) binary predictions (0/1) or logits
39         gt_boundary: (N, H, W) binary ground truth (0/1)
40     """
41     if pred_boundary.dtype != torch.bool:
42         pred_boundary = (torch.sigmoid(pred_boundary) > 0.5).float()
43     pred_flat = pred_boundary.cpu().numpy().flatten().astype(np.int32)
44     gt_flat = gt_boundary.cpu().numpy().flatten().astype(np.int32)
45     return f1_score(gt_flat, pred_flat, zero_division=0)
46
47
48 def measure_fps(model, input_size=(360, 480), device=DEVICE, num_warmup=10, num_runs=100):
49     """Measure inference FPS on the given device."""
50     model.eval()
51     dummy = torch.randn(1, 3, *input_size).to(device)
52
53     # Warmup
54     with torch.no_grad():
55         for _ in range(num_warmup):
56             _ = model(dummy)
57
58     # Timed runs
59     torch.cuda.synchronize() if device.type == "cuda" else None
60     t0 = time.time()
61     with torch.no_grad():

```

```
62     for _ in range(num_runs):  
63         _ = model(dummy)
```